

TypeScript Interview Questions — 50 Questions (Full-Stack Focus)

📖 **How to use this file:** You already know types and interfaces. This set skips the baby stuff and goes straight to what interviewers actually ask — generics, utility types, narrowing, mapped types, and real patterns. Every question has: what it is → why it matters → real code → memory trick.

Section 1: Type System Foundations (Q1–8)

You "know" these but interviews go deeper. Make sure you can explain the WHY, not just the HOW.

Q1. What is the difference between `type` and `interface`? When do you use each?

Both define the shape of an object. But they are NOT the same thing.

`interface` — for describing object shapes (and can be extended/merged)

```
interface User {
  id: number;
  name: string;
  email: string;
}

// Extend with another interface
interface AdminUser extends User {
  role: "admin";
  permissions: string[];
}
```

Declaration merging — interfaces with the same name AUTO-MERGE:

```
interface Window {
  myCustomProp: string;
}

// TypeScript merges this with the built-in Window interface
// Useful for extending third-party types
```

`type` — for anything an interface can't do

```
// Union types (interface can't do this)
type Status = "active" | "inactive" | "pending";

// Intersection (combining types)
type AdminUser = User & { role: "admin" };

// Tuple
type Coordinates = [number, number];

// Computed / conditional types
type ID = string | number;

// Function type
type Handler = (event: MouseEvent) => void;
```

Side-by-side

Feature	<code>interface</code>	<code>type</code>
---------	------------------------	-------------------

Object shapes Feature	interface	type
Union types	❌ No	✅ Yes
Declaration merging	✅ Yes	❌ No
extends keyword	✅ Yes	✅ (use & instead)
Computed types	❌ No	✅ Yes
Clearer error messages	✅ Usually	⚠ Sometimes verbose

The real-world rule most teams follow

```
// Use interface for: objects, classes, React props, API shapes
interface UserProps {
  name: string;
  onClick: () => void;
}
```

```
// Use type for: unions, utility combos, primitives, tuples
type Theme = "light" | "dark";
type ApiResponse<T> = { data: T; error: string | null };
```

❗ **Memory trick:** `interface` = a contract for an object's shape. Can be extended and merged. `type` = an alias for ANYTHING. More flexible, can't be merged. When in doubt about objects → `interface`. When you need union/computed → `type`.

Q2. What is `unknown` vs `any`? Why is `unknown` safer?

Both mean "I don't know the type yet." But they behave very differently.

`any` — opt out of TypeScript completely

```
let value: any = "hello";

value.toUpperCase(); // ❌ no error – but what if value is a number?
value.nonExistent(); // ❌ no error – TypeScript just gives up checking
value = 42;          // ❌ no error
value = { name: "x" }; // ❌ no error

// TypeScript trusts you completely – even when you're wrong
// No protection at all
```

`unknown` — safe version of `any`

```
let value: unknown = "hello";

value.toUpperCase(); // ❌ ERROR – TypeScript won't allow this
value.nonExistent(); // ❌ ERROR – must narrow first

// You MUST check the type before using it
if (typeof value === "string") {
  value.toUpperCase(); // ✅ Now TypeScript knows it's a string
}
```

`unknown` forces you to PROVE the type before you use it. This prevents bugs.

Real world — handling API responses safely

```
// ❌ Bad — using any loses all type safety
async function fetchUser(): Promise<any> {
  const res = await fetch("/api/user");
  return res.json(); // could be anything — TypeScript won't warn you
}

// ✅ Good — using unknown forces you to validate
async function fetchUser(): Promise<unknown> {
  const res = await fetch("/api/user");
  const data: unknown = await res.json();

  // Must check before using
  if (isUser(data)) {
    return data; // TypeScript knows it's a User here
  }

  throw new Error("Invalid user data");
}

function isUser(value: unknown): value is User {
  return (
    typeof value === "object" &&
    value !== null &&
    "id" in value &&
    "name" in value
  );
}
```

never — the third special type

While we're here — `never` means "this can never happen":

```
function throwError(message: string): never {
  throw new Error(message);
  // never returns — so return type is never
}

// Useful in exhaustive checks (covered in Q15)
```

Type	Meaning	Use when
<code>any</code>	Could be anything — TypeScript gives up	Migrating old JS (avoid if possible)
<code>unknown</code>	Could be anything — must check first	API responses, user input, JSON parsing
<code>never</code>	Can never happen	Exhaustive checks, functions that always throw

❗ **Memory trick:** `any` = blindfolded driver (goes wherever, no safety) `unknown` = cautious driver who checks the road before going `never` = a road that doesn't exist

Q3. What is a Union type and an Intersection type? Show real examples.

Union type `A | B` — can be ONE of these types

"Either this OR that."

```

type ID = string | number;

let userId: ID = "user_abc"; // ✅
userId = 123;                // ✅
userId = true;                // ❌ ERROR

// Real world – function that accepts both
function getUser(id: string | number) {
  if (typeof id === "string") {
    return db.users.findBySlug(id);
  }
  return db.users.findById(id);
}

```

```

// Union with literal types – very common in React
type ButtonVariant = "primary" | "secondary" | "danger";
type Status = "idle" | "loading" | "success" | "error";

function Button({ variant }: { variant: ButtonVariant }) {
  // TypeScript knows variant can only be one of these 3 values
}

```

Intersection type **A & B** – must have ALL properties of both

"This AND that."

```

interface HasName {
  name: string;
}

interface HasEmail {
  email: string;
}

// Must have BOTH name AND email
type UserContact = HasName & HasEmail;

const contact: UserContact = {
  name: "Zain",
  email: "zain@example.com",
  // ❌ Missing email would be an error
};

```

```

// Real world – adding metadata to any type
type WithTimestamps = {
  createdAt: Date;
  updatedAt: Date;
};

type Post = {
  title: string;
  content: string;
};

type PostWithTimestamps = Post & WithTimestamps;
// Must have: title, content, createdAt, updatedAt

```

Union A | B

Intersection A & B

Meaning	One OR the other Union A B	Both combined Intersection A & B
Properties	Has properties of EITHER	Has properties of ALL
<code>extends</code> equivalent	N/A	Similar to <code>extends</code>
Use case	"This can be X or Y"	"This is X with extra Y added"

📌 **Memory trick:** Union `|` = OR gate. Either/or. At least one. Intersection `&` = AND gate. Everything combined. Must have all.

Q4. What is `readonly` and `const` in TypeScript? How are they different?

`const` — for variables that can't be reassigned

```
const name = "Zain";
name = "Ali"; // ❌ ERROR — can't reassign a const

// BUT — object contents can still change
const user = { name: "Zain", age: 25 };
user.name = "Ali"; // ✅ allowed! const only prevents reassignment
user = {}; // ❌ ERROR — can't reassign the variable
```

`readonly` — for object properties that can't be changed

```
interface User {
  readonly id: number; // ← can never be changed after creation
  name: string; // ← can be changed
}

const user: User = { id: 1, name: "Zain" };
user.name = "Ali"; // ✅ fine
user.id = 99; // ❌ ERROR — id is readonly
```

```
// readonly array — can't push, pop, or modify
const ids: readonly number[] = [1, 2, 3];
ids.push(4); // ❌ ERROR
ids[0] = 99; // ❌ ERROR
```

```
// Same as:
const ids2: ReadonlyArray<number> = [1, 2, 3];
```

as `const` — make everything deeply `readonly` + literal types

```
const config = {
  endpoint: "/api/users",
  method: "GET",
  timeout: 3000,
} as const;

// Without as const:
// method is typed as: string

// With as const:
// method is typed as: "GET" (the literal string, not just any string)
// Everything is readonly

config.method = "POST"; // ❌ ERROR – readonly
```

Real use case — action types in Redux/reducers:

```
const ACTION_TYPES = {
  LOGIN: "auth/login",
  LOGOUT: "auth/logout",
  UPDATE: "auth/update",
} as const;

type ActionType = typeof ACTION_TYPES[keyof typeof ACTION_TYPES];
// → "auth/login" | "auth/logout" | "auth/update"
```

❗ **Memory trick:** `const` = the box can't be replaced (but contents can change) `readonly` = the contents of the box can't be changed `as const` = freeze everything — box and contents

Q5. What is a Tuple? When is it useful?

A tuple is an array with a **fixed number of elements** where each position has a **specific type**.

Regular array: all elements are the same type, any length. Tuple: exact length, each position has its own type.

```
// Regular array — all numbers, any length
const nums: number[] = [1, 2, 3, 4, 5];

// Tuple — exactly 2 elements: first is string, second is number
const nameAge: [string, number] = ["Zain", 25];
nameAge[0].toUpperCase(); // ❗ TypeScript knows index 0 is a string
nameAge[1].toFixed(2);     // ❗ TypeScript knows index 1 is a number

const [name, age] = nameAge; // destructuring works naturally
```

Real world use cases

useState return value is a tuple:

```
// React's useState returns exactly [value, setter]
const [count, setCount] = useState(0);
// count: number
// setCount: Dispatch<SetStateAction<number>>
```

Custom hook return value:

```
// Returns [data, loading, error] – exact positions matter
function useApi(url: string): [User | null, boolean, string | null] {
  const [data, setData] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  // ... fetch logic

  return [data, loading, error];
}

// Usage:
const [user, isLoading, errorMsg] = useApi("/api/user/1");
```

Named tuples (clearer):

```
// Named tuples – positions have names for clarity
type Range = [start: number, end: number];
type RGB = [red: number, green: number, blue: number];

const color: RGB = [255, 128, 0];
```

🧠 **Memory trick:** Tuple = "fixed-size array with a contract." Position 0 is always X, position 1 is always Y. Like a pair of shoes – always left and right, in that order.

Q6. What is `typeof`, `keyof`, and `instanceof` in TypeScript?

Three powerful operators – very commonly asked in interviews.

`typeof` – get the type of a VALUE

```
const user = {
  id: 1,
  name: "Zain",
  email: "zain@example.com",
};

// Instead of writing the interface manually, infer it from the object
type User = typeof user;
// → { id: number; name: string; email: string }

// Works on functions too
function createPost(title: string, content: string) {
  return { title, content, createdAt: new Date() };
}

type Post = ReturnType<typeof createPost>;
// → { title: string; content: string; createdAt: Date }
```

`keyof` – get all property names of a type as a union

```

interface User {
  id: number;
  name: string;
  email: string;
}

type UserKeys = keyof User;
// → "id" | "name" | "email"

// Real use – safe property access
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key]; // TypeScript knows this is safe
}

const user: User = { id: 1, name: "Zain", email: "z@z.com" };
getProperty(user, "name"); // 🔍 returns string
getProperty(user, "id");   // 🔍 returns number
getProperty(user, "missing"); // 🔍 ERROR – not a key of User

```

instanceof — check if a value is an instance of a class

```

class ApiError extends Error {
  constructor(public statusCode: number, message: string) {
    super(message);
  }
}

class NetworkError extends Error {
  constructor(public retryAfter: number) {
    super("Network failed");
  }
}

function handleError(error: unknown) {
  if (error instanceof ApiError) {
    // TypeScript knows error has statusCode here
    console.log(error.statusCode);
  } else if (error instanceof NetworkError) {
    // TypeScript knows error has retryAfter here
    console.log(error.retryAfter);
  }
}

```

Operator	Input	Output	Use for
<code>typeof x</code>	A value	Its type	Infer type from runtime value
<code>keyof T</code>	A type	Union of its keys	Safe property access
<code>instanceof</code>	A value + class	boolean	Narrow class instances

🔍 **Memory trick:** `typeof` = "what type IS this value?" `keyof` = "what KEYS does this type have?" `instanceof` = "is this an instance of that class?"

Q7. What is a Literal Type? What is a Template Literal Type?

Literal type — a variable that can only be ONE specific value

```
// Instead of string, the type IS the specific string
type Direction = "north" | "south" | "east" | "west";
type HTTPMethod = "GET" | "POST" | "PUT" | "DELETE" | "PATCH";
type DiceRoll = 1 | 2 | 3 | 4 | 5 | 6;

function move(direction: Direction) {
  // direction can ONLY be one of the 4 values
}

move("north"); // ✅
move("up");    // ❌ ERROR – "up" is not a Direction
```

Template Literal Type — build string types with patterns

Like JavaScript template literals, but at the type level.

```
type EventName = "click" | "focus" | "blur";

// Creates: "onClick" | "onFocus" | "onBlur"
type HandlerName = `on${Capitalize<EventName>}`;

// Real world – API endpoint patterns
type UserId = `user_${string}`;
const id: UserId = "user_abc123"; // ✅
const bad: UserId = "admin_abc";  // ❌ ERROR – must start with "user_"
```

```
// Build all combinations
type Color = "red" | "blue";
type Size = "sm" | "md" | "lg";

type ColorSize = `${Color}-${Size}`;
// → "red-sm" | "red-md" | "red-lg" | "blue-sm" | "blue-md" | "blue-lg"
```

```
// Real use in Next.js – typed route params
type PostRoute = `/blog/${string}`;
type UserRoute = `/users/${number}`;
```

🧠 **Memory trick:** Literal type = a type that IS the value, not just describes it. Template literal type = regex for types — describe patterns of strings.

Q8. What is `Optional Chaining (?.)` and `Non-null Assertion (!)`? When should you use each?

Optional chaining `?.` — safely access nested properties

```

interface User {
  name: string;
  address?: {
    city?: string;
    zip?: string;
  };
}

const user: User = { name: "Zain" }; // no address

// ❌ Crash – address is undefined
console.log(user.address.city); // TypeError: Cannot read properties of undefined

// ✅ Safe – returns undefined instead of crashing
console.log(user.address?.city); // → undefined
console.log(user.address?.city?.toUpperCase()); // → undefined (no crash)

// Works with methods and arrays too
user.getProfile?.(); // call only if method exists
users?.[0]?.name; // safe array access

```

Non-null assertion `!` — tell TypeScript "trust me, it's not null"

```

// TypeScript thinks getElementById might return null
const button = document.getElementById("submit-btn");
button.click(); // ❌ ERROR – button could be null

// You tell TypeScript: "I know this exists"
const button = document.getElementById("submit-btn")!;
button.click(); // ✅ TypeScript trusts you

// Common in React refs
const inputRef = useRef<HTMLInputElement>(null);
inputRef.current!.focus(); // you know it's mounted, assert it's not null

```

When to use each

Situation	Use
Property MIGHT not exist	<code>?.</code> optional chaining
You're 100% sure it's not null	<code>!</code> non-null assertion
Not sure?	Use <code>?.</code> — safer

```

// ❌ Overusing ! is dangerous
const user = getUser()!; // what if getUser() actually returns null?
user.name; // runtime crash – TypeScript was silenced

// ✅ Better – check first
const user = getUser();
if (user) {
  user.name; // safe
}

```

❗ **Memory trick:** `?.` = "if it exists, continue — otherwise return undefined" `!` = "I promise TypeScript this isn't null — stop warning me" Prefer `?.` — it's defensive. Use `!` only when you're absolutely certain.

Section 2: Generics (Q9–16)

The most important TypeScript topic for interviews. Understand this deeply.

Q9. What is a Generic? Why do we need them? Explain with a simple example.

The problem without generics

```
// Without generics – you'd need separate functions for every type
function getFirstString(arr: string[]): string {
  return arr[0];
}

function getFirstNumber(arr: number[]): number {
  return arr[0];
}

function getFirstUser(arr: User[]): User {
  return arr[0];
}

// ☹️ Copying the same function for every type – not maintainable
```

The solution — Generics

A generic is a **type variable** — a placeholder for a type that gets filled in when used.

```
// ONE function that works for ANY type
function getFirst<T>(arr: T[]): T {
  return arr[0];
}
//           ↑   ↑       ↑
//           T is declared here
//           T is used here (array of T)
//           T is the return type

// TypeScript INFERS T from what you pass in:
getFirst([1, 2, 3]);           // T = number → returns number
getFirst(["a", "b", "c"]);     // T = string → returns string
getFirst([{ name: "Zain" }]);  // T = { name: string } → returns that
```

How to read generics

```
function identity<T>(value: T): T {
  return value;
}
```

Read it as: "This function takes a type parameter T. It takes a value of type T and returns a value of type T."

T is just a convention — you can name it anything:

- T = Type (generic, most common)
- K = Key
- V = Value
- E = Element
- TData, TError — more descriptive names

Real world — generic API response wrapper

```
// Without generic – you'd need ApiResponse for every type
interface UserApiResponse {
  data: User;
  error: string | null;
  loading: boolean;
}

// With generic – ONE type for all responses
interface ApiResponse<T> {
  data: T | null;
  error: string | null;
  loading: boolean;
}

type UserResponse = ApiResponse<User>;
// → { data: User | null; error: string | null; loading: boolean }

type PostsResponse = ApiResponse<Post[]>;
// → { data: Post[] | null; error: string | null; loading: boolean }
```

📌 **Memory trick:** Generic = a type with a blank to fill in later. Like a form template: "Name: ____" – the blank gets filled when you use it. `Array<T>` is a generic – it's an array of SOME type. `Array<string>` fills in the blank.

Q10. What are Generic Constraints? What does `extends` do inside a generic?

Without constraints, `T` can be ANYTHING. Sometimes you need `T` to have certain properties – that's a constraint.

```
// Problem – T could be anything, including a number
function printName<T>(value: T) {
  console.log(value.name); // ❌ ERROR – T might not have .name
}

// Solution – constrain T to types that have a name property
function printName<T extends { name: string }>(value: T) {
  console.log(value.name); // ✅ TypeScript knows T has name
}

printName({ name: "Zain", age: 25 }); // ✅
printName({ name: "Post", views: 100 }); // ✅
printName(42); // ❌ ERROR – number has no name
```

`K extends keyof T` – the most common constraint pattern

```
// Safe property access – K must be a key of T
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const user = { id: 1, name: "Zain", email: "z@z.com" };

getProperty(user, "name"); // ✅ returns string
getProperty(user, "id"); // ✅ returns number
getProperty(user, "missing"); // ❌ ERROR – "missing" is not a key of user
```

Multiple constraints

```
// T must extend both HasId AND HasName
function processItem<T extends HasId & HasName>(item: T) {
  console.log(`Processing ${item.id}: ${item.name}`);
}
```

Default type parameter

```
// T defaults to string if not specified
function createList<T = string>(): T[] {
  return [];
}

const strings = createList();           // T = string (default)
const numbers = createList<number>(); // T = number (explicit)
```

📌 **Memory trick:** `T extends Something` = "T must be at least this type." It's like a job requirement: "Applicant must have at least 2 years experience." The applicant (T) can have MORE, but must meet the minimum.

Q11. Write a generic `useState`-style hook. Write a generic fetch function.

Generic custom hook

```
import { useState, useCallback } from "react";

// Generic hook – works with any type T
function useToggle<T>(onValue: T, offValue: T): [T, () => void] {
  const [value, setValue] = useState<T>(offValue);

  const toggle = useCallback(() => {
    setValue(prev => prev === onValue ? offValue : onValue);
  }, [onValue, offValue]);

  return [value, toggle];
}

// Usage:
const [isOpen, toggleOpen] = useToggle(true, false);
const [theme, toggleTheme] = useToggle("dark", "light");
const [status, toggleStatus] = useToggle("active", "inactive");
```

Generic fetch function – the most important real-world pattern

```
// Without generic – returns any (not safe)
async function fetchData(url: string): Promise<any> {
  const res = await fetch(url);
  return res.json();
}

// With generic – caller specifies what they expect back
async function fetchData<T>(url: string): Promise<T> {
  const res = await fetch(url);

  if (!res.ok) {
    throw new Error(`HTTP error: ${res.status}`);
  }

  return res.json() as Promise<T>;
}

// Usage – TypeScript knows exactly what comes back
const user = await fetchData<User>("/api/user/1");
// user is typed as User 🎯

const posts = await fetchData<Post[]>("/api/posts");
// posts is typed as Post[] 🎯

user.name;      // 🎯 TypeScript knows this exists
posts[0].title; // 🎯 TypeScript knows this exists
```

Even better — with error handling and response wrapper

```
interface FetchResult<T> {
  data: T | null;
  error: string | null;
}

async function safeFetch<T>(url: string): Promise<FetchResult<T>> {
  try {
    const res = await fetch(url);

    if (!res.ok) {
      return { data: null, error: `HTTP ${res.status}` };
    }

    const data: T = await res.json();
    return { data, error: null };
  } catch (err) {
    return { data: null, error: "Network error" };
  }
}

// Usage
const { data: user, error } = await safeFetch<User>("/api/user/1");
if (error) console.log(error);
if (user) console.log(user.name); // TypeScript knows user is User here
```

🧠 **Memory trick:** Generic function = a function that says "tell me the type when you call me." `fetchData<User>(url)` = "fetch this URL and I promise the result is a User."

Q12. What is a Generic Class? Show a typed Stack or Repository pattern.

Generic Stack

```
class Stack<T> {
  private items: T[] = [];

  push(item: T): void {
    this.items.push(item);
  }

  pop(): T | undefined {
    return this.items.pop();
  }

  peek(): T | undefined {
    return this.items[this.items.length - 1];
  }

  get size(): number {
    return this.items.length;
  }

  isEmpty(): boolean {
    return this.items.length === 0;
  }
}

// A stack of numbers
const numberStack = new Stack<number>();
numberStack.push(1);
numberStack.push(2);
numberStack.pop(); // returns number

// A stack of strings
const stringStack = new Stack<string>();
stringStack.push("hello");
```

Generic Repository pattern — real full-stack pattern

```
// Base repository that works with any entity that has an id
interface Entity {
  id: string;
}

class Repository<T extends Entity> {
  private items: Map<string, T> = new Map();

  findById(id: string): T | undefined {
    return this.items.get(id);
  }

  findAll(): T[] {
    return Array.from(this.items.values());
  }

  save(item: T): T {
    this.items.set(item.id, item);
    return item;
  }

  delete(id: string): boolean {
    return this.items.delete(id);
  }
}

// Reuse for any entity
interface User extends Entity {
  name: string;
  email: string;
}

interface Post extends Entity {
  title: string;
  content: string;
}

const userRepo = new Repository<User>();
const postRepo = new Repository<Post>();

userRepo.save({ id: "1", name: "Zain", email: "z@z.com" });
const user = userRepo.findById("1"); // typed as User | undefined
```

🧠 **Memory trick:** Generic class = a class with a type blank. `Stack<T>` = "a stack of SOMETHING" — you fill in the something when creating it.

Q13. What is a Generic Interface? How do you use it in a React component?

```

// Generic interface
interface SelectProps<T> {
  options: T[];
  value: T | null;
  onChange: (value: T) => void;
  getLabel: (option: T) => string;
  getValue: (option: T) => string | number;
}

// Generic React component
function Select<T>({
  options,
  value,
  onChange,
  getLabel,
  getValue,
}: SelectProps<T>) {
  return (
    <select
      value={value ? String(getValue(value)) : ""}
      onChange={(e) => {
        const selected = options.find(
          (opt) => String(getValue(opt)) === e.target.value
        );
        if (selected) onChange(selected);
      }}
    >
      {options.map((option) => (
        <option key={String(getValue(option))} value={String(getValue(option))}>
          {getLabel(option)}
        </option>
      ))}
    </select>
  );
}

// Usage – works with ANY type
interface Country {
  code: string;
  name: string;
}

<Select<Country>
  options={countries}
  value={selectedCountry}
  onChange={setSelectedCountry}
  getLabel={(c) => c.name}
  getValue={(c) => c.code}
/>

```

Q14. What is `infer` in TypeScript? Show a practical example.

`infer` is used inside conditional types to **extract** a type from another type.

It says: "if this type matches this pattern, grab THIS part of it and call it ____."

Built-in uses of `infer` (you use these without knowing it)

```
// ReturnType – extracts the return type of a function
type ReturnType<T> = T extends (...args: any[]) => infer R ? R : never;
//                                     ↑ infer R = "grab the return type"

function getUser() {
  return { id: 1, name: "Zain" };
}

type User = ReturnType<typeof getUser>;
// → { id: number; name: string }
```

Writing your own with `infer`

```
// Extract the type inside a Promise
type UnwrapPromise<T> = T extends Promise<infer Value> ? Value : T;

type A = UnwrapPromise<Promise<string>>; // → string
type B = UnwrapPromise<Promise<User>>;   // → User
type C = UnwrapPromise<string>;          // → string (not a Promise, returns T itself)

// Extract the first argument type of a function
type FirstArgument<T> = T extends (first: infer F, ...args: any[]) => any ? F : never;

function login(email: string, password: string) {}

type EmailType = FirstArgument<typeof login>; // → string
```

Real use — extract array item type

```
type ArrayItem<T> = T extends (infer Item)[] ? Item : never;

type A = ArrayItem<string[]>; // → string
type B = ArrayItem<User[]>;   // → User
type C = ArrayItem<number>;    // → never (not an array)
```

📌 **Memory trick:** `infer R` = "when this pattern matches, capture THIS piece and name it R." Like a regex capture group — you match a pattern and pull out part of it.

Q15. What is an Exhaustive Check with `never`? Why is it important?

An exhaustive check ensures you handle EVERY case of a union type. If you add a new case to the union later, TypeScript FORCES you to handle it.

```

type Shape = "circle" | "square" | "triangle";

function getArea(shape: Shape): number {
  switch (shape) {
    case "circle":
      return Math.PI * 5 * 5;
    case "square":
      return 5 * 5;
    case "triangle":
      return 0.5 * 5 * 5;
    default:
      // If we handled all cases, shape is `never` here
      // If we MISSED a case, TypeScript gives an error
      const _exhaustiveCheck: never = shape;
      throw new Error(`Unhandled shape: ${shape}`);
  }
}

```

Why this is powerful:

```

// Later, someone adds "hexagon" to the union
type Shape = "circle" | "square" | "triangle" | "hexagon";

// Now TypeScript gives an ERROR in the default case:
// "Type 'hexagon' is not assignable to type 'never'"
// → Forces you to add the hexagon case

```

Without the exhaustive check, the new case would silently fall through to `default` and return undefined.

🧠 **Memory trick:** `never` in a switch default = "if I got here, something impossible happened." If TypeScript complains about the `never` assignment → you missed a case. Go fix it.

Q16. Explain `ReturnType<T>`, `Parameters<T>`, and `InstanceType<T>`.

These are built-in generic utility types that extract info about functions and classes.

```

// ReturnType<T> – what does this function return?
function createUser(name: string) {
  return { id: Math.random(), name, createdAt: new Date() };
}

type User = ReturnType<typeof createUser>;
// → { id: number; name: string; createdAt: Date }

// Useful when you don't control the function but need its return type
type NextPageProps = ReturnType<typeof getServerSideProps>;

```

```

// Parameters<T> – what arguments does this function take?
function login(email: string, password: string, rememberMe: boolean) {}

type LoginParams = Parameters<typeof login>;
// → [string, string, boolean]

type EmailParam = Parameters<typeof login>[0];
// → string (first parameter)

```

```
// InstanceType<T> – what does `new ClassName()` produce?
class UserService {
  getUser(id: string) { return { id, name: "Zain" }; }
  createUser(name: string) { return { id: "1", name }; }
}

type UserServiceInstance = InstanceType<typeof UserService>;
// → UserService (the instance type)

// Useful when class is passed as a value
function createService<T>(ServiceClass: new () => T): T {
  return new ServiceClass();
}
```

🔖 **Memory trick:** `ReturnType` = "what comes OUT of the function" `Parameters` = "what goes INTO the function" `InstanceType` = "what do you GET when you new this class"

Section 3: Type Narrowing & Guards (Q17–22)

Q17. What is Type Narrowing? Explain all the ways to narrow a type.

Type narrowing = helping TypeScript understand that a wide type (like `string | number`) is actually a more specific type in a certain code block.

TypeScript is smart — it tracks type information through your code. The more you narrow, the more it knows.

1. `typeof` narrowing

```
function process(value: string | number) {
  if (typeof value === "string") {
    // TypeScript knows: value is string here
    return value.toUpperCase();
  }
  // TypeScript knows: value is number here
  return value.toFixed(2);
}
```

2. `instanceof` narrowing

```
function handleError(error: Error | ApiError) {
  if (error instanceof ApiError) {
    // TypeScript knows: error is ApiError here
    console.log(error.statusCode);
  } else {
    // TypeScript knows: error is Error here
    console.log(error.message);
  }
}
```

3. `in` operator narrowing

```
interface Cat { meow(): void; }
interface Dog { bark(): void; }

function makeSound(animal: Cat | Dog) {
  if ("meow" in animal) {
    animal.meow(); // TypeScript knows: animal is Cat
  } else {
    animal.bark(); // TypeScript knows: animal is Dog
  }
}
```

4. Truthiness narrowing

```
function greet(name: string | null | undefined) {
  if (name) {
    // TypeScript knows: name is string here (not null/undefined)
    return `Hello, ${name.toUpperCase()}`;
  }
  return "Hello, stranger";
}
```

5. Equality narrowing

```
function compare(a: string | number, b: string | boolean) {
  if (a === b) {
    // TypeScript knows: both must be string (the only type they share)
    a.toUpperCase(); // ✖
    b.toUpperCase(); // ✖
  }
}
```

6. Discriminated union narrowing (most important — see Q18)

```
type Action =
  | { type: "LOGIN"; email: string }
  | { type: "LOGOUT" }
  | { type: "UPDATE"; field: string; value: string };

function handle(action: Action) {
  switch (action.type) {
    case "LOGIN":
      console.log(action.email); // TypeScript knows .email exists
      break;
    case "LOGOUT":
      // TypeScript knows no extra fields
      break;
    case "UPDATE":
      console.log(action.field, action.value); // TypeScript knows both exist
      break;
  }
}
```

✖ **Memory trick:** Narrowing = TypeScript tracking your if/switch checks to get smarter about types. Every `if (typeof x === "string")` check makes TypeScript smarter inside that block.

Q18. What is a Discriminated Union? Why is it the best pattern for state management?

A discriminated union is a union where each member has a **shared literal property** (the discriminant) that identifies which member it is.

```
// Each member has "type" as the discriminant
type RequestState<T> =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; error: string };
```

TypeScript uses the `status` field to know which shape you're working with:

```
function renderState(state: RequestState<User>) {
  switch (state.status) {
    case "idle":
      return <p>Click to load</p>;

    case "loading":
      return <p>Loading...</p>;

    case "success":
      // TypeScript knows state.data exists here ☑
      return <p>Welcome, {state.data.name}</p>;

    case "error":
      // TypeScript knows state.error exists here ☑
      return <p>Error: {state.error}</p>;
  }
}
```

Why it's better than boolean flags

```
// ☑ Boolean flags – impossible states are representable
interface BadState {
  isLoading: boolean;
  isError: boolean;
  data: User | null;
  error: string | null;
  // What does isLoading: true, isError: true even mean?
  // How many combinations of these 4 fields are there? 2^4 = 16
  // Most of them are nonsensical
}

// ☑ Discriminated union – only valid states exist
type GoodState =
  | { status: "loading" }
  | { status: "error"; error: string }
  | { status: "success"; data: User };
// Only 3 states. Each is valid. No impossible combinations.
```

☑ **Memory trick:** Discriminated union = a union where one shared field tells you which shape you're dealing with. The discriminant is the field TypeScript uses as the "this is which type it is" signal. Best pattern for: API request states, form states, multi-step flows.

Q19. What is a Type Guard? Write a custom one.

A type guard is a function that **proves** to TypeScript what type a value is.

It uses a special return type: `value is Type` – called a **type predicate**.

```

// The magic return type: "value is User"
// Means: "if this function returns true, TypeScript should treat value as User"
function isUser(value: unknown): value is User {
  return (
    typeof value === "object" &&
    value !== null &&
    "id" in value &&
    "name" in value &&
    typeof (value as User).id === "number" &&
    typeof (value as User).name === "string"
  );
}

// Usage
const data: unknown = await fetch("/api/user").then(r => r.json());

if (isUser(data)) {
  // TypeScript knows data is User here ☑️
  console.log(data.name.toUpperCase());
  console.log(data.id + 1);
}

```

Type guard for discriminated union

```

type Circle = { kind: "circle"; radius: number };
type Square = { kind: "square"; side: number };
type Shape = Circle | Square;

function isCircle(shape: Shape): shape is Circle {
  return shape.kind === "circle";
}

function getArea(shape: Shape): number {
  if (isCircle(shape)) {
    // TypeScript knows shape is Circle ☑️
    return Math.PI * shape.radius ** 2;
  }
  // TypeScript knows shape is Square ☑️
  return shape.side ** 2;
}

```

Type guard for API responses (most practical)

```

interface ApiUser {
  id: number;
  name: string;
  email: string;
}

function isApiUser(value: unknown): value is ApiUser {
  if (typeof value !== "object" || value === null) return false;

  const obj = value as Record<string, unknown>;

  return (
    typeof obj.id === "number" &&
    typeof obj.name === "string" &&
    typeof obj.email === "string"
  );
}

// Now you can safely type API responses
const response: unknown = await fetchUser();
if (!isApiUser(response)) {
  throw new Error("Invalid API response shape");
}
// TypeScript knows response is ApiUser from here ☑

```

☑ **Memory trick:** Type guard = a function that acts as proof for TypeScript. `value is User` in the return type = "if I return true, I PROMISE it's a User." TypeScript trusts that promise and narrows the type inside the `if` block.

Q20. What is the difference between `as` (type assertion) and a Type Guard?

Type Assertion `as` — you force TypeScript to believe you

```

const value: unknown = "hello";

// You TELL TypeScript what type it is — no check performed
const str = value as string;
str.toUpperCase(); // TypeScript allows it — but what if value was a number?

```

`as` is a **lie you tell TypeScript**. If you're wrong, it crashes at runtime.

```

// Dangerous
const user = JSON.parse(rawJson) as User;
user.name.toUpperCase(); // ☑ crashes if JSON didn't actually have a name property

```

Type Guard — TypeScript verifies at runtime

```

// TypeScript checks at runtime + narrows at compile time
if (isUser(data)) {
  data.name.toUpperCase(); // ☑ safe — isUser proved it
}

```

Double assertion `as unknown as T` — a red flag

```

// This is a code smell — usually means something is wrong with your types
const evil = someValue as unknown as User;
// Bypasses all type checking — only do this if you know EXACTLY what you're doing

```

	as (assertion)	Type Guard
Runtime check?	❌ No	✅ Yes
Safe?	⚠️ Only if you're right	✅ Yes
TypeScript trusts you	Blindly	After you prove it
Use when	You know more than TS	API responses, user input

❗ **Memory trick:** `as` = "TypeScript, trust me, I know what this is." (pinky promise, no verification) `Type guard` = "TypeScript, let me PROVE it to you." (shows ID)

Q21. What is `satisfies`? How is it different from a type annotation?

Added in TypeScript 4.9. One of the most useful newer features.

The problem

```
const config: Record<string, string | number> = {
  host: "localhost",
  port: 3000,
};

// TypeScript knows config.host is string | number (the wide type)
// NOT just string – so you lose autocompletion and narrowing
config.host.toUpperCase(); // ❌ ERROR – might be number
```

`satisfies` – validate the type WITHOUT widening it

```
const config = {
  host: "localhost",
  port: 3000,
} satisfies Record<string, string | number>;

// TypeScript VALIDATES the shape against Record<string, string | number>
// BUT keeps the inferred types of each property

config.host.toUpperCase(); // ✅ TypeScript knows host is string (not string | number)
config.port.toFixed(2);    // ✅ TypeScript knows port is number
```

Real use case – typed color palette

```
type Colors = Record<string, string | { light: string; dark: string }>;

const palette = {
  red: "#ff0000",
  blue: { light: "#add8e6", dark: "#00008b" },
  green: "#00ff00",
} satisfies Colors;

// Without satisfies: palette.blue is string | { light: string; dark: string }
// With satisfies:    palette.blue is { light: string; dark: string } ← specific!

palette.blue.light; // ✅ TypeScript knows .light exists
```

❗ **Memory trick:** Regular annotation: "treat this AS that type" – you lose the specific info. `satisfies`: "CHECK this against that type, but KEEP the specific info."
`satisfies` = validator. Annotation = cast.

Q22. What is Assertion Functions? How do they work?

An assertion function throws if a condition is false — and TypeScript narrows the type after the call.

```
// Regular type guard (returns boolean)
function isString(value: unknown): value is string {
    return typeof value === "string";
}

// Assertion function (throws if wrong)
// Return type: "asserts value is string"
function assertIsString(value: unknown): asserts value is string {
    if (typeof value !== "string") {
        throw new Error(`Expected string, got ${typeof value}`);
    }
}

function processInput(input: unknown) {
    assertIsString(input); // throws if not string

    // TypeScript now knows input is string
    console.log(input.toUpperCase());
}
```

asserts condition — assert a boolean condition

```
function assert(condition: boolean, message: string): asserts condition {
    if (!condition) {
        throw new Error(message);
    }
}

const user: User | null = getUser();
assert(user !== null, "User must not be null");

// TypeScript knows user is User here (not null)
console.log(user.name);
```

🔗 **Memory trick:** Assertion function = "I promise TypeScript this is true, or I'll throw an error." After the call, TypeScript trusts the narrowing — because if it was wrong, execution stopped.

Section 4: Utility Types (Q23–30)

Q23. Explain `Partial<T>`, `Required<T>`, and `Readonly<T>`.

These transform an existing type — make all fields optional, required, or readonly.

`Partial<T>` — make ALL properties optional

```
interface User {
  id: number;
  name: string;
  email: string;
  bio: string;
}

type PartialUser = Partial<User>;
// → { id?: number; name?: string; email?: string; bio?: string }

// Real use – update function (you only pass what changed)
async function updateUser(id: number, updates: Partial<User>) {
  await db.users.update(id, updates);
}

updateUser(1, { name: "New Name" }); // ✖ only name
updateUser(1, { email: "new@email.com" }); // ✖ only email
updateUser(1, { name: "Zain", bio: "Developer" }); // ✖ multiple fields
```

Required<T> — make ALL properties required (removes optional)

```
interface Config {
  host?: string;
  port?: number;
  timeout?: number;
}

type FinalConfig = Required<Config>;
// → { host: string; port: number; timeout: number }

// Real use – after validating/filling defaults, assert everything is present
function buildConfig(partial: Config): Required<Config> {
  return {
    host: partial.host ?? "localhost",
    port: partial.port ?? 3000,
    timeout: partial.timeout ?? 5000,
  };
}
```

Readonly<T> — make ALL properties readonly

```
interface User {
  id: number;
  name: string;
}

type ImmutableUser = Readonly<User>;
// → { readonly id: number; readonly name: string }

const user: Readonly<User> = { id: 1, name: "Zain" };
user.name = "Ali"; // ✖ ERROR – cannot assign to readonly property

// Real use – props in React (you shouldn't mutate props)
function UserCard({ user }: { user: Readonly<User> }) {
  user.name = "Ali"; // ✖ ERROR – good, props should never be mutated
  return <div>{user.name}</div>;
}
```

✖ **Memory trick:** `Partial` = put `?` on everything (all optional) `Required` = remove all `?` (all mandatory) `Readonly` = put `readonly` on everything (freeze it)

Q24. Explain `Pick<T, K>`, `Omit<T, K>`, and `Exclude<T, U>`.

`Pick<T, K>` — keep ONLY the listed properties

```
interface User {
  id: number;
  name: string;
  email: string;
  password: string;
  createdAt: Date;
}

// Only keep id, name, email — exclude password and createdAt
type PublicUser = Pick<User, "id" | "name" | "email">;
// → { id: number; name: string; email: string }

// Real use — API response (never send password to client)
async function getPublicUser(id: number): Promise<PublicUser> {
  const user = await db.users.findById(id);
  return { id: user.id, name: user.name, email: user.email };
}
```

`Omit<T, K>` — remove the listed properties, keep everything else

```
// Remove password and createdAt — keep everything else
type PublicUser = Omit<User, "password" | "createdAt">;
// → { id: number; name: string; email: string }

// Pick vs Omit:
// Pick = "I want THESE fields" (whitelist)
// Omit = "I want everything EXCEPT these" (blacklist)

// Omit is better when you have many fields and only want to remove a few
// Pick is better when you only want a small subset
```

`Exclude<T, U>` — remove types from a union (not properties!)

```
type Status = "active" | "inactive" | "banned" | "deleted";

// Remove "banned" and "deleted" from the union
type VisibleStatus = Exclude<Status, "banned" | "deleted">;
// → "active" | "inactive"

// Real use
type NonNullable<T> = Exclude<T, null | undefined>;
// (this is actually a built-in TypeScript utility)

type MaybeString = string | null | undefined;
type DefiniteString = Exclude<MaybeString, null | undefined>;
// → string
```

Utility	Works on	What it does
<code>Pick<T, K></code>	Object type	Keep only listed PROPERTIES
<code>Omit<T, K></code>	Object type	Remove listed PROPERTIES
<code>Exclude<T, U></code>	Union type	Remove listed TYPES from union

Utility **looks on** **What it does:**
The `omit` trick: Pick `Omit` to pick what you want. `Omit` = omit/skip what you don't want. `Exclude` = exclude types from a union (note: it's for UNIONS not objects)

Q25. Explain `Record<K, V>`, `Extract<T, U>`, and `NonNullable<T>`.

`Record<K, V>` — create a type with specific key-value pairs

```
// Record<KeyType, ValueType>

// All string keys, number values
type ScoreBoard = Record<string, number>;
const scores: ScoreBoard = {
  zain: 95,
  ali: 87,
  sara: 92,
};

// Specific keys
type UserRoles = Record<"admin" | "user" | "moderator", boolean>;
const roles: UserRoles = {
  admin: true,
  user: false,
  moderator: true,
};

// Real use — index objects by ID
type UserMap = Record<string, User>;
const usersById: UserMap = {
  "user_1": { id: 1, name: "Zain" },
  "user_2": { id: 2, name: "Ali" },
};
```

`Extract<T, U>` — opposite of `Exclude` (keep only matching types)

```
type AllEvents = "click" | "focus" | "blur" | "keydown" | "keyup";

// Keep only keyboard events
type KeyboardEvents = Extract<AllEvents, "keydown" | "keyup">;
// → "keydown" | "keyup"

// Extract objects matching a shape
type Shape = { kind: "circle" } | { kind: "square" } | { kind: "triangle" };
type RoundShapes = Extract<Shape, { kind: "circle" }>;
// → { kind: "circle" }
```

`NonNullable<T>` — remove null and undefined

```
type MaybeUser = User | null | undefined;

type DefiniteUser = NonNullable<MaybeUser>;
// → User

// Real use — after null check
function processUser(user: User | null) {
  if (!user) return;
  // Here user is NonNullable<User | null> = User
  processDefiniteUser(user); // TypeScript knows user is User
}
```

❗ **Memory trick:** `Record<K,V>` = a typed dictionary/map `Extract` = keep only what matches (opposite of `Exclude`) `NonNullable` = "remove null and undefined, trust it's there"

Q26. What is `ReturnType`, and how do you use it to keep types in sync?

```
// When you have a function that returns a complex object
// Instead of manually defining the return type, infer it

function buildUserConfig(userId: string) {
  return {
    userId,
    permissions: ["read", "write"] as string[],
    settings: {
      theme: "dark" as "dark" | "light",
      notifications: true,
    },
    createdAt: new Date(),
  };
}

// Derive the type from the function — stays in sync automatically
type UserConfig = ReturnType<typeof buildUserConfig>;

// Now UserConfig is always correct — even if buildUserConfig changes
function applyConfig(config: UserConfig) {
  console.log(config.settings.theme.toUpperCase()); // ❗
}
```

Practical pattern — keep API types in sync with handlers

```
// Route handler
async function GET(req: Request) {
  const users = await db.users.findAll();
  return Response.json({
    users,
    total: users.length,
    page: 1,
  });
}

// The frontend can derive the response type
type UsersResponse = Awaited<ReturnType<typeof GET>>;
// Always matches what the server actually returns ❗
```

❗ **Memory trick:** `ReturnType` = "don't write the type twice — let TypeScript figure it out from the function." One source of truth — change the function, the type updates automatically.

Q27. What is a Conditional Type? Write a real example.

A conditional type is a type-level ternary: `T extends U ? TypeIfTrue : TypeIfFalse`

```
// Is T a string? If yes → "string type". If no → "not string type"
type IsString<T> = T extends string ? "string type" : "not string type";

type A = IsString<string>; // → "string type"
type B = IsString<number>; // → "not string type"
type C = IsString<"hello">; // → "string type" (literal string extends string)
```

Real example — flatten array types

```
// If T is an array, give me what's inside it. Otherwise, give me T.
type Flatten<T> = T extends (infer Item)[] ? Item : T;

type A = Flatten<string[]>; // → string
type B = Flatten<number[]>; // → number
type C = Flatten<string>;    // → string (not array, return as-is)
```

Distributive conditional types

When T is a union, conditional types apply to EACH member:

```
type ToArray<T> = T extends any ? T[] : never;

type A = ToArray<string | number>;
// → string[] | number[]
// Applied to each member: string → string[], number → number[]
```

Real use — make a type nullable only in certain contexts

```
type MakeNullable<T, IsNullable extends boolean> =
  IsNullable extends true ? T | null : T;

type A = MakeNullable<string, true>; // → string | null
type B = MakeNullable<string, false>; // → string
```

🧠 **Memory trick:** Conditional type = ternary for types. `T extends U ? Yes : No` = "does T fit into U? If yes, give Yes type. If no, give No type."

Q28. What are Mapped Types? Build a real one from scratch.

A mapped type loops over every key in a type and transforms it.

```
// Syntax: { [K in keyof T]: ... }
// "For every key K in T, create a property..."

// Make all properties optional (this is how Partial<T> is built!)
type MyPartial<T> = {
  [K in keyof T]?: T[K];
  //      ↑      ↑
  //      make optional   keep original type
};

// Make all properties readonly (this is how Readonly<T> is built!)
type MyReadonly<T> = {
  readonly [K in keyof T]: T[K];
};

// Make all properties nullable
type Nullable<T> = {
  [K in keyof T]: T[K] | null;
};

interface User {
  id: number;
  name: string;
  email: string;
}

type NullableUser = Nullable<User>;
// → { id: number | null; name: string | null; email: string | null }
```

Key remapping with `as`

```
// Add "get" prefix to all keys
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K];
};

type UserGetters = Getters<User>;
// → { getId: () => number; getName: () => string; getEmail: () => string }
```

Filter properties by type

```
// Keep only string properties
type StringProperties<T> = {
  [K in keyof T as T[K] extends string ? K : never]: T[K];
};

interface Mixed {
  name: string;
  age: number;
  email: string;
  active: boolean;
}

type StringOnly = StringProperties<Mixed>;
// → { name: string; email: string }
```

▮ **Memory trick:** Mapped type = `for each key in T, do something`. Like `Array.map()` but for object TYPES instead of array values.

Q29. What is `Awaited<T>`? Why do you need it with async functions?

`Awaited<T>` recursively unwraps a Promise type.

```
type A = Awaited<Promise<string>>;
// → string

type B = Awaited<Promise<Promise<number>>>>;
// → number (unwraps nested Promises)

type C = Awaited<string>;
// → string (not a Promise, returns as-is)
```

Real use — typing async function return values

```
async function fetchUser() {
  return { id: 1, name: "Zain", email: "z@z.com" };
}

// Without Awaited:
type A = ReturnType<typeof fetchUser>;
// → Promise<{ id: number; name: string; email: string }>
// Still wrapped in Promise — not the actual data type

// With Awaited:
type B = Awaited<ReturnType<typeof fetchUser>>;
// → { id: number; name: string; email: string }
// The actual resolved type 🎯
```

In Next.js — typing page props

```
// Next.js generateMetadata is async
export async function generateMetadata({ params }: PageProps) {
  const post = await fetchPost(params.slug);
  return { title: post.title };
}

type MetadataResult = Awaited<ReturnType<typeof generateMetadata>>;
// → { title: string }
```

🧠 **Memory trick:** `Awaited<T>` = "unwrap the Promise, give me what's inside." Like `await` at the type level — removes the Promise wrapper.

Q30. What is `Parameters<T>` and how do you use it to type event handlers?

```
// Extract parameter types from a function
function createUser(name: string, email: string, role: "admin" | "user") {
  return { name, email, role };
}

type CreateUserParams = Parameters<typeof createUser>;
// → [string, string, "admin" | "user"]

type NameParam = Parameters<typeof createUser>[0]; // → string
type RoleParam = Parameters<typeof createUser>[2]; // → "admin" | "user"
```

Real use — typing event handlers consistently

```
// Extract the event type from an event handler
type ButtonClickHandler = (event: React.MouseEvent<HTMLButtonElement>) => void;
type InputChangeHandler = (event: React.ChangeEvent<HTMLInputElement>) => void;

// Or derive from existing handlers
function handleSubmit(event: React.FormEvent<HTMLFormElement>) {
  event.preventDefault();
}

type SubmitParams = Parameters<typeof handleSubmit>;
// → [React.FormEvent<HTMLFormElement>]

type SubmitEvent = Parameters<typeof handleSubmit>[0];
// → React.FormEvent<HTMLFormElement>
```

Wrapping existing functions safely

```
// You have a third-party function
function thirdPartyUpload(file: File, bucket: string, options: UploadOptions) {}

// Create a wrapper with the same parameter types – stays in sync
function uploadWithRetry(...args: Parameters<typeof thirdPartyUpload>) {
  try {
    return thirdPartyUpload(...args);
  } catch {
    return thirdPartyUpload(...args); // retry
  }
}
// If thirdPartyUpload changes its signature, uploadWithRetry updates automatically
```

📌 **Memory trick:** `Parameters<T>` = "give me the argument types of this function as a tuple." Use it when you want to wrap or re-use a function's signature without duplicating types.

Section 5: Classes, Decorators & Advanced Patterns (Q31–38)

Q31. How do TypeScript classes differ from JavaScript classes?

TypeScript adds access modifiers, abstract classes, and proper typing on top of JS classes.

Access modifiers

```
class UserService {
  public name: string;      // accessible everywhere (default)
  private password: string; // only inside this class
  protected role: string;   // inside this class + subclasses
  readonly id: number;       // can't be changed after construction

  constructor(name: string, password: string, id: number) {
    this.name = name;
    this.password = password;
    this.id = id;
    this.role = "user";
  }

  // Shorthand – declare AND assign in constructor params
  // constructor(
  //   public name: string,
  //   private password: string,
  //   readonly id: number
  // ) {}
}

const service = new UserService("Zain", "secret", 1);
service.name;      // ✅ public
service.password;  // ❌ ERROR – private
service.id = 99;   // ❌ ERROR – readonly
```

Parameter shorthand (most common in interviews)

```
class Post {
  // Declare + assign in constructor – very clean
  constructor(
    public readonly id: string,
    public title: string,
    private content: string,
    protected authorId: string,
  ) {}

  getSummary() {
    return `${this.title} (${this.content.slice(0, 100)}...)`;
  }
}
```

Abstract classes

```
// Can't be instantiated directly – must be subclassed
abstract class BaseRepository<T> {
  abstract findById(id: string): Promise<T | null>;
  abstract findAll(): Promise<T[]>;
  abstract save(entity: T): Promise<T>;

  // Concrete method available to all subclasses
  async findOrFail(id: string): Promise<T> {
    const result = await this.findById(id);
    if (!result) throw new Error(`Not found: ${id}`);
    return result;
  }
}

class UserRepository extends BaseRepository<User> {
  async findById(id: string) { /* implement */ return null; }
  async findAll() { /* implement */ return []; }
  async save(user: User) { /* implement */ return user; }
}
```

📌 **Memory trick:** `public` = anyone can use it `private` = only I can use it `protected` = me and my children can use it `abstract` = I define the SHAPE, subclass fills in the CONTENT

Q32. What are Decorators in TypeScript? Show a real example.

Decorators are functions that wrap classes, methods, or properties to add behavior. They use `@` syntax and run at class definition time (not at instantiation).

Note: Requires `"experimentalDecorators": true` in `tsconfig.json`

Method decorator — most common

```
// A decorator is just a function
function Log(target: any, propertyKey: string, descriptor: PropertyDescriptor) {
  const originalMethod = descriptor.value;

  descriptor.value = async function (...args: any[]) {
    console.log(`Calling ${propertyKey} with:`, args);
    const result = await originalMethod.apply(this, args);
    console.log(`${propertyKey} returned:`, result);
    return result;
  };

  return descriptor;
}

class UserService {
  @Log // ← decorates the method below
  async getUser(id: string) {
    return db.users.findById(id);
  }

  @Log
  async createUser(data: CreateUserInput) {
    return db.users.create(data);
  }
}

// Every call to getUser or createUser is now automatically logged
```

Class decorator

```
function Injectable(target: new (...args: any[]) => any) {
  // Register class in a dependency injection container
  container.register(target);
}

@Injectable
class EmailService {
  sendEmail(to: string, subject: string) { }
}
```

Property decorator — validation

```
function MinLength(min: number) {
  return function(target: any, propertyKey: string) {
    let value: string;

    Object.defineProperty(target, propertyKey, {
      get: () => value,
      set: (newVal: string) => {
        if (newVal.length < min) {
          throw new Error(` ${propertyKey} must be at least ${min} chars`);
        }
        value = newVal;
      },
    });
  };
}

class User {
  @MinLength(3)
  name: string = "";
}
```

🧠 **Memory trick:** Decorator = a wrapper that adds behavior without changing the original code. `@Log` before a method = "wrap this method with logging." Like a label on a package — it doesn't change the contents, just adds info/behavior on the outside.

Q33. How do you type `this` in TypeScript classes and functions?

`this` parameter in functions

```
interface User {
  name: string;
  greet(this: User): string; // 'this' must be a User when called
}

function greet(this: User): string {
  return `Hello, I am ${this.name}`;
}

const user: User = { name: "Zain", greet };
user.greet(); // ❌

const greetFn = user.greet;
greetFn(); // ❌ ERROR — 'this' would not be a User here (it's undefined/window)
```

Fluent interface / method chaining (common pattern)

```

class QueryBuilder {
  private conditions: string[] = [];
  private limitValue?: number;

  where(condition: string): this { // returns 'this' for chaining
    this.conditions.push(condition);
    return this;
  }

  limit(n: number): this {
    this.limitValue = n;
    return this;
  }

  build(): string {
    const where = this.conditions.length
      ? `WHERE ${this.conditions.join(" AND ")}`
      : "";
    const limit = this.limitValue ? `LIMIT ${this.limitValue}` : "";
    return `SELECT * FROM users ${where} ${limit}`.trim();
  }
}

const query = new QueryBuilder()
  .where("age > 18")
  .where("active = true")
  .limit(10)
  .build();
// "SELECT * FROM users WHERE age > 18 AND active = true LIMIT 10"

```

📌 **Memory trick:** this parameter = tell TypeScript what `this` must be when calling this function. Returning `this` from methods = enables method chaining. TypeScript tracks it correctly.

Q34. What is Declaration Merging? When is it useful?

Declaration merging = when TypeScript combines multiple declarations with the same name.

Interface merging — add properties to existing interfaces

```

// Original interface (maybe from a library)
interface Request {
  url: string;
  method: string;
}

// You extend it in your project (in a .d.ts file)
interface Request {
  user?: User; // ← added
  sessionId?: string; // ← added
}

// TypeScript merges both – Request now has all 4 properties
function middleware(req: Request) {
  console.log(req.user?.name); // 📌 TypeScript knows about user
}

```

Extending third-party types (most common real use)

```
// Extending Express Request type in express.d.ts
declare global {
  namespace Express {
    interface Request {
      user?: User;
    }
  }
}

// Now in route handlers:
app.get("/dashboard", (req, res) => {
  console.log(req.user?.name); // 📌 TypeScript knows about req.user
});
```

Extending Window in browser code

```
// global.d.ts
interface Window {
  analytics: {
    track: (event: string, data?: object) => void;
  };
}

// Now anywhere in your code:
window.analytics.track("page_view"); // 📌 TypeScript knows this exists
```

📌 **Memory trick:** Declaration merging = "add more to an existing type without editing the original." Most useful for extending third-party types you can't edit.

Q35. What is a `namespace`? When would you use it over a module?

```
// Namespace – a named container for related types/values
namespace Validation {
  export interface StringValidator {
    isAcceptable(s: string): boolean;
  }

  export function validate(value: string): boolean {
    return value.length > 0;
  }

  export const MIN_LENGTH = 3;
}

// Usage
const result = Validation.validate("hello");
const isValid: Validation.StringValidator = { isAcceptable: (s) => s.length > 0 };
```

When to use namespace vs module

```
// Use namespace: grouping related types in declaration files (.d.ts)
namespace API {
  export interface UserEndpoint {
    GET: { response: User };
    POST: { body: CreateUserInput; response: User };
  }
}

// In modern TypeScript: prefer MODULES over namespaces
// Namespaces are mainly used in:
// 1. Declaration files (.d.ts) for typing JS libraries
// 2. Organizing types in a single file
// 3. Legacy code

// Modern alternative – just use a regular module
export namespace API {
  export interface User { id: number; name: string; }
}
```

📌 **Memory trick:** Namespace = a bucket to put related things in. Modern TypeScript prefers ES modules (`import / export`). Namespaces appear mainly in `.d.ts` files and older codebases.

Q36. How do you type React components in TypeScript? Show props, children, and generics.

Basic typed component

```
interface ButtonProps {
  label: string;
  onClick: () => void;
  variant?: "primary" | "secondary" | "danger";
  disabled?: boolean;
}

function Button({ label, onClick, variant = "primary", disabled = false }: ButtonProps) {
  return (
    <button
      onClick={onClick}
      disabled={disabled}
      className={`btn btn-${variant}`}
    >
      {label}
    </button>
  );
}
```

Typing children

```
import { ReactNode, PropsWithChildren } from "react";

// Option 1 – explicit
interface CardProps {
  title: string;
  children: ReactNode; // ← anything React can render
}

// Option 2 – PropsWithChildren helper
type CardProps = PropsWithChildren<{
  title: string;
}>;
// → { title: string; children?: ReactNode }

function Card({ title, children }: CardProps) {
  return (
    <div className="card">
      <h2>{title}</h2>
      {children}
    </div>
  );
}
```

Generic component

```
interface ListProps<T> {
  items: T[];
  renderItem: (item: T, index: number) => ReactNode;
  keyExtractor: (item: T) => string;
}

function List<T>({ items, renderItem, keyExtractor }: ListProps<T>) {
  return (
    <ul>
      {items.map((item, index) => (
        <li key={keyExtractor(item)}>{renderItem(item, index)}</li>
      ))}
    </ul>
  );
}

// Usage – works with any type
<List
  items={users}
  renderItem={(user) => <span>{user.name}</span>}
  keyExtractor={(user) => user.id.toString()}
/>
```

Typing forwardRef

```
import { forwardRef } from "react";

interface InputProps {
  label: string;
  value: string;
  onChange: (value: string) => void;
}

const Input = forwardRef<HTMLInputElement, InputProps>(
  ({ label, value, onChange }, ref) => {
    return (
      <div>
        <label>{label}</label>
        <input
          ref={ref}
          value={value}
          onChange={(e) => onChange(e.target.value)}
        />
      </div>
    );
  }
);
```

📌 **Memory trick:** React props = just an interface. children: ReactNode = anything React can render. Generic component = `function List<T>` — same as generic function but returns JSX.

Q37. What are Declaration Files (`.d.ts`)? When do you write them?

A `.d.ts` file contains **only type information** — no runtime code. TypeScript uses them to know the types of code it can't see (JS files, external packages).

When you'd write one

1. Typing a plain JavaScript file/library:

```
// math.js (no types)
function add(a, b) { return a + b; }
function multiply(a, b) { return a * b; }

// math.d.ts (types for math.js)
export function add(a: number, b: number): number;
export function multiply(a: number, b: number): number;
```

2. Declaring global variables (like environment globals):

```
// globals.d.ts
declare const __DEV__: boolean;
declare const __VERSION__: string;
declare const __API_URL__: string;
```

3. Extending third-party types (declaration merging):

```
// express.d.ts
import { User } from "../models/user";

declare global {
  namespace Express {
    interface Request {
      user?: User;
    }
  }
}
```

4. Typing a CSS/image import:

```
// images.d.ts
declare module "*.png" {
  const src: string;
  export default src;
}

declare module "*.svg" {
  const ReactComponent: React.FC<React.SVGProps<SVGSVGElement>>;
  export { ReactComponent };
  const src: string;
  export default src;
}
```

🔗 **Memory trick:** `.d.ts` = a type-only file. No JavaScript runs from it. Like a shadow — it has the exact same shape as the JS file, but it's just types.

Q38. How do you type environment variables in Next.js with TypeScript?

Without typing, `process.env.ANYTHING` is `string | undefined` — no autocomplete, easy to misspell.

```
// env.d.ts — type your environment variables
namespace NodeJS {
  interface ProcessEnv {
    // Required (string, not undefined)
    DATABASE_URL: string;
    JWT_SECRET: string;
    NEXTAUTH_SECRET: string;

    // Optional
    REDIS_URL?: string;

    // Public (available in browser)
    NEXT_PUBLIC_API_URL: string;
    NEXT_PUBLIC_APP_NAME: string;

    // Environment
    NODE_ENV: "development" | "test" | "production";
  }
}
```

Now in your code:

```
// Before: process.env.DATABASE_URL is string | undefined
// After: process.env.DATABASE_URL is string (autocomplete works!)

const db = new Database(process.env.DATABASE_URL); // 🚫 no undefined warning
const apiUrl = process.env.NEXT_PUBLIC_API_URL;    // 🚫 autocomplete works
```

Validate at startup with Zod (production pattern)

```
// lib/env.ts
import { z } from "zod";

const envSchema = z.object({
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
  NEXT_PUBLIC_API_URL: z.string().url(),
  NODE_ENV: z.enum(["development", "test", "production"]),
});

// Validates at startup – app crashes with clear message if env var is missing
export const env = envSchema.parse(process.env);

// Usage (fully typed + validated)
import { env } from "@lib/env";
const db = new Database(env.DATABASE_URL); // 🔍 typed, validated, autocomplete
```

🔒 **Memory trick:** Extend `NodeJS.ProcessEnv` in a `.d.ts` file = give your env vars proper types. Use Zod to validate them at startup = app fails fast with a clear error, not a cryptic crash later.

Section 6: Real-World Patterns (Q39–50)

Q39. How do you type a REST API response end-to-end (backend → frontend)?

```

// shared/types/api.ts – shared between backend and frontend
export interface User {
  id: number;
  name: string;
  email: string;
  role: "admin" | "user";
  createdAt: string; // ISO date string from JSON
}

export interface ApiResponse<T> {
  data: T;
  message: string;
  success: boolean;
}

export interface PaginatedResponse<T> extends ApiResponse<T[]> {
  pagination: {
    page: number;
    limit: number;
    total: number;
    totalPages: number;
  };
}

// Backend – app/api/users/route.ts
import { PaginatedResponse, User } from "@shared/types/api";

export async function GET(): Promise<Response> {
  const users = await db.users.findAll();
  const response: PaginatedResponse<User> = {
    data: users,
    message: "Users fetched successfully",
    success: true,
    pagination: { page: 1, limit: 10, total: users.length, totalPages: 1 },
  };
  return Response.json(response);
}

// Frontend
async function getUsers(): Promise<PaginatedResponse<User>> {
  const res = await fetch("/api/users");
  return res.json() as Promise<PaginatedResponse<User>>;
}

```

Q40. How do you type a Zustand or Redux store with TypeScript?

Typed Zustand store

```

import { create } from "zustand";

interface User {
  id: number;
  name: string;
  email: string;
}

interface AuthState {
  user: User | null;
  isAuthenticated: boolean;
  isLoading: boolean;
  login: (email: string, password: string) => Promise<void>;
  logout: () => void;
}

const useAuthStore = create<AuthState>((set) => ({
  user: null,
  isAuthenticated: false,
  isLoading: false,

  login: async (email, password) => {
    set({ isLoading: true });
    try {
      const user = await authService.login(email, password);
      set({ user, isAuthenticated: true, isLoading: false });
    } catch {
      set({ isLoading: false });
      throw new Error("Login failed");
    }
  },

  logout: () => {
    set({ user: null, isAuthenticated: false });
  },
}));

// Usage – fully typed
const { user, login, logout } = useAuthStore();
user?.name; // user is User | null – TypeScript knows

```

Q41. How do you write a typed custom hook that returns data, loading, and error?

```

import { useState, useEffect } from "react";

interface FetchState<T> {
  data: T | null;
  loading: boolean;
  error: string | null;
}

function useFetch<T>(url: string): FetchState<T> {
  const [state, setState] = useState<FetchState<T>>>({
    data: null,
    loading: true,
    error: null,
  });

  useEffect(() => {
    let cancelled = false;

    async function fetchData() {
      setState(prev => ({ ...prev, loading: true, error: null }));

      try {
        const res = await fetch(url);
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const data: T = await res.json();

        if (!cancelled) {
          setState({ data, loading: false, error: null });
        }
      } catch (err) {
        if (!cancelled) {
          setState({
            data: null,
            loading: false,
            error: err instanceof Error ? err.message : "Unknown error",
          });
        }
      }
    }

    fetchData();
    return () => { cancelled = true; }; // cleanup – prevent state update on unmount
  }, [url]);

  return state;
}

// Usage
const { data: user, loading, error } = useFetch<User>("/api/user/1");
if (loading) return <Spinner />;
if (error) return <Error message={error} />;
if (user) return <UserCard name={user.name} />;

```

Q42. How do you type a form with React Hook Form and TypeScript?

```

import { useForm, SubmitHandler } from "react-hook-form";
import { z } from "zod";
import { zodResolver } from "@hookform/resolvers/zod";

// Define schema with Zod
const loginSchema = z.object({
  email: z.string().email("Invalid email"),
  password: z.string().min(8, "Password must be at least 8 characters"),
  rememberMe: z.boolean().optional(),
});

// Derive TypeScript type from schema – single source of truth
type LoginFormData = z.infer<typeof loginSchema>;
// → { email: string; password: string; rememberMe?: boolean }

function LoginForm() {
  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting },
  } = useForm<LoginFormData>({
    resolver: zodResolver(loginSchema),
  });

  const onSubmit: SubmitHandler<LoginFormData> = async (data) => {
    // data is fully typed as LoginFormData
    await loginUser(data.email, data.password);
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register("email")} />
      {errors.email && <p>{errors.email.message}</p>}

      <input {...register("password")} type="password" />
      {errors.password && <p>{errors.password.message}</p>}

      <button type="submit" disabled={isSubmitting}>Login</button>
    </form>
  );
}

```

Q43. How do you handle `JSON.parse` safely with TypeScript?

```

// ⚠ Unsafe – JSON.parse returns any
const user = JSON.parse(rawJson); // any – no type safety
user.name; // ⚠ TypeScript won't warn even if this crashes

// ⚠ Option 1 – use unknown and type guard
function parseJSON<T>(json: string): T | null {
  try {
    const parsed: unknown = JSON.parse(json);
    return parsed as T; // still needs a type guard ideally
  } catch {
    return null;
  }
}

// ⚠ Option 2 – use Zod to validate AND type (best pattern)
import { z } from "zod";

const UserSchema = z.object({
  id: z.number(),
  name: z.string(),
  email: z.string().email(),
});

function parseUser(json: string) {
  const parsed: unknown = JSON.parse(json);
  return UserSchema.parse(parsed); // throws if invalid, returns typed User if valid
}

// Usage
const user = parseUser(rawJson); // fully typed, validated at runtime
user.name.toUpperCase(); // ⚠ safe

```

Q44. What is `tsconfig.json`? Explain the most important options.

```

{
  "compilerOptions": {
    // TypeScript version target
    "target": "ES2020",          // what JS version to output

    // Module system
    "module": "ESNext",          // use modern import/export
    "moduleResolution": "bundler", // how to resolve imports (Next.js uses this)

    // Strictness – ALWAYS enable in new projects
    "strict": true,              // enables all strict checks below:
    //   strictNullChecks        → null/undefined must be handled explicitly
    //   noImplicitAny            → can't use any without being explicit
    //   strictFunctionTypes      → stricter function type checking

    // Paths (import aliases)
    "baseUrl": ".",
    "paths": {
      "@/*": [".src/*"]          // import from "@components/..." instead of "../../.."
    },

    // JSX
    "jsx": "preserve",           // let Next.js handle JSX transform

    // Output
    "noEmit": true,              // don't output JS files (Next.js handles this)

    // Extra safety
    "noUnusedLocals": true,      // error on unused variables
    "noUnusedParameters": true, // error on unused function params
    "exactOptionalPropertyTypes": true, // undefined ≠ missing
  }
}

```

Most important option: `"strict": true`

```

// With strict: false (dangerous defaults)
function greet(name) { // name is implicitly any – ❗ no warning
  return name.toUpperCase();
}

// With strict: true
function greet(name: string) { // must be explicit ❗
  return name.toUpperCase();
}

```

❗ **Memory trick:** `strict: true` = TypeScript on hard mode. Always use it on new projects. Without strict, TypeScript is barely better than JavaScript.

Q45. How do you type a Next.js Server Action with input validation?

```

// app/actions/user.ts
"use server";

import { z } from "zod";
import { revalidatePath } from "next/cache";

// Input schema
const CreateUserSchema = z.object({
  name: z.string().min(2).max(50),
  email: z.string().email(),
  role: z.enum(["admin", "user"]),
});

type CreateUserInput = z.infer<typeof CreateUserSchema>;

// Action return type
type ActionResult =
  | { success: true; user: User }
  | { success: false; error: string; fieldErrors?: Record<string, string[]> };

export async function createUser(formData: FormData): Promise<ActionResult> {
  // Parse and validate
  const rawData = {
    name: formData.get("name"),
    email: formData.get("email"),
    role: formData.get("role"),
  };

  const result = CreateUserSchema.safeParse(rawData);

  if (!result.success) {
    return {
      success: false,
      error: "Validation failed",
      fieldErrors: result.error.flatten().fieldErrors,
    };
  }

  try {
    const user = await db.users.create(result.data);
    revalidatePath("/admin/users");
    return { success: true, user };
  } catch {
    return { success: false, error: "Failed to create user" };
  }
}

```

Q46. What is `omit` vs `Exclude` — and when do people mix them up?

This is one of the most common TypeScript mistakes in interviews.

```
// Omit – removes PROPERTIES from an OBJECT TYPE
interface User {
  id: number;
  name: string;
  password: string;
}

type PublicUser = Omit<User, "password">;
// → { id: number; name: string }
// ⓘ Used on INTERFACES/OBJECT TYPES

// Exclude – removes TYPES from a UNION
type Status = "active" | "inactive" | "banned";
type AllowedStatus = Exclude<Status, "banned">;
// → "active" | "inactive"
// ⓘ Used on UNION TYPES

// ⓘ Common mistake – using Exclude on an object type
type Wrong = Exclude<User, "password">;
// → User (nothing happens – User is not a union, "password" is not a union member)

// ⓘ Common mistake – using Omit on a union type
type Wrong2 = Omit<Status, "banned">;
// → {} (completely wrong – Status is a union, not an object)
```

	Omit<T, K>	Exclude<T, U>
Input	Object/Interface	Union type
Removes	Properties	Types from union
Output	Object type	Union type
Mistake	Using on union	Using on object

ⓘ **Memory trick:** `Omit` = remove PROPERTIES (from objects, like omitting columns from a table) `Exclude` = remove TYPES (from unions, like excluding items from a list) Object → Omit. Union → Exclude.

Q47. What is `satisfies` vs `as const` vs type annotation? When to use each?

```

const palette1 = {
  red: "#ff0000",
  blue: "#0000ff",
};
// Inferred type: { red: string; blue: string }
// Flexible but no autocomplete on values

const palette2: Record<string, string> = {
  red: "#ff0000",
  blue: "#0000ff",
};
// Type is widened to Record<string, string>
// palette2.red is string – lost the specific value
// Can add any string key

const palette3 = {
  red: "#ff0000",
  blue: "#0000ff",
} as const;
// Type: { readonly red: "#ff0000"; readonly blue: "#0000ff" }
// Values are literals – can't be changed
// Can't add new keys

const palette4 = {
  red: "#ff0000",
  blue: "#0000ff",
} satisfies Record<string, string>;
// Validates shape as Record<string, string>
// BUT keeps specific types: red is "#ff0000" not just string
// Best of both worlds ☺

```

	Annotation	as const	satisfies
Validates against type	☒	☒	☒
Keeps specific types	☒ (widens)	☒ (literals)	☒
Mutable	☒	☒ (readonly)	☒
Use for	General typing	Freeze + literals	Validate + keep specifics

Q48. How do you type error handling in TypeScript? What about typed errors?

```

// TypeScript 4.0+ – catch errors as unknown (safest)
async function riskyOperation() {
  try {
    const result = await fetchData();
    return result;
  } catch (error: unknown) {
    // error is unknown – must narrow before using

    if (error instanceof Error) {
      console.log(error.message); // ☒ TypeScript knows .message exists
      console.log(error.stack);   // ☒
    }

    if (error instanceof ApiError) {
      console.log(error.statusCode); // ☒ typed custom error
    }

    throw error; // re-throw if you can't handle it
  }
}

```

```

    }
}

// Custom typed error classes
class ApiError extends Error {
  constructor(
    public readonly statusCode: number,
    public readonly code: string,
    message: string,
  ) {
    super(message);
    this.name = "ApiError";
  }
}

class ValidationError extends Error {
  constructor(
    public readonly fields: Record<string, string[]>,
    message: string,
  ) {
    super(message);
    this.name = "ValidationError";
  }
}

// Result type pattern – no exceptions
type Result<T, E = Error> =
  | { ok: true; value: T }
  | { ok: false; error: E };

async function safeLogin(email: string, password: string): Promise<Result<User>> {
  try {
    const user = await authService.login(email, password);
    return { ok: true, value: user };
  } catch (error) {
    return {
      ok: false,
      error: error instanceof Error ? error : new Error("Unknown error"),
    };
  }
}

// Usage – no try/catch needed at call site
const result = await safeLogin(email, password);
if (result.ok) {
  console.log(result.value.name); // typed as User
} else {
  console.log(result.error.message); // typed as Error
}

```

Q49. What is the difference between `interface extends` and intersection types & ?

Both combine types, but they behave differently when there are conflicts.

```

interface A {
  value: string;
}

interface B {
  value: number; // ← conflicts with A.value
}

// interface extends – ERROR on conflict
interface C extends A, B {}
// ❌ TypeScript error: Types of property 'value' are incompatible

// Intersection – makes conflicting property NEVER
type D = A & B;
// D.value is: string & number = never
// Still compiles but value can never be assigned!

const d: D = { value: "hello" }; // ❌ string is not assignable to never

```

When to prefer which

```

// Use extends for clean inheritance with no conflicts
interface Animal {
  name: string;
}

interface Dog extends Animal {
  breed: string;
}

// Use & for composing independent types
type AdminUser = User & { permissions: string[] };
type LoggedRequest = Request & { userId: string; timestamp: Date };

```

❗ **Memory trick:** `extends` = clean inheritance, errors on conflict (safer) `&` = merge everything, conflict becomes `never` (sneaky bugs) When uncertain: use `extends`. Use `&` for simple additions.

Q50. TypeScript config for a production Next.js full-stack app. What would you set?

```
{
  "compilerOptions": {
    "target": "ES2020",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": false,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [{ "name": "next" }],
    "baseUrl": ".",
    "paths": {
      "@/*": ["./*"]
    },
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,
    "exactOptionalPropertyTypes": true
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules"]
}
```

The key decisions explained:

"strict": true	→ non-negotiable. Catches 90% of bugs.
"noUnusedLocals": true	→ catches dead code
"noUnusedParameters": true	→ catches dead function params
"exactOptionalPropertyTypes"	→ { a?: string } means a is string undefined, not that a can be MISSING or undefined (stricter than default)
"moduleResolution": "bundler"	→ correct for Next.js + Vite
"isolatedModules": true	→ each file compiled independently (required for Babel/SWC)
"skipLibCheck": true	→ skip type checking of .d.ts files in node_modules (speeds up compile, most lib types are correct anyway)

🔑 **Final memory trick:** `strict: true` is the single most important setting. Everything else is tuning. Start strict, loosen specific rules only if you have a good reason.

🔑 Full Set Summary

Section 1: Type System Foundations

- Q1. type vs interface
- Q2. unknown vs any vs never
- Q3. Union | vs Intersection &
- Q4. readonly vs const vs as const
- Q5. Tuples
- Q6. typeof, keyof, instanceof
- Q7. Literal types + template literal types
- Q8. Optional chaining ?. and non-null assertion !

Section 2: Generics

- Q9. What is a Generic + why we need them
- Q10. Generic constraints (extends)
- Q11. Generic fetch function + useState hook
- Q12. Generic class (Stack + Repository)
- Q13. Generic React component
- Q14. infer keyword
- Q15. Exhaustive check with never
- Q16. ReturnType, Parameters, InstanceType

Section 3: Type Narrowing & Guards

- Q17. All narrowing techniques
- Q18. Discriminated unions
- Q19. Custom type guards
- Q20. Type assertion as vs type guards
- Q21. satisfies operator
- Q22. Assertion functions

Section 4: Utility Types

- Q23. Partial, Required, Readonly
- Q24. Pick, Omit, Exclude
- Q25. Record, Extract, NonNullable
- Q26. ReturnType for sync between types
- Q27. Conditional types
- Q28. Mapped types
- Q29. Awaited
- Q30. Parameters

Section 5: Classes, Decorators & Advanced Patterns

- Q31. TypeScript classes (access modifiers, abstract)
- Q32. Decorators
- Q33. this keyword + fluent interfaces
- Q34. Declaration merging
- Q35. Namespaces
- Q36. Typed React components (props, children, generics, forwardRef)
- Q37. Declaration files .d.ts
- Q38. Typing environment variables

Section 6: Real-World Patterns

- Q39. Typing REST API end-to-end
- Q40. Typed Zustand store
- Q41. Generic custom hook (data/loading/error)
- Q42. React Hook Form + Zod + TypeScript
- Q43. Safe JSON.parse
- Q44. tsconfig.json important options
- Q45. Server Action with typed validation
- Q46. Omit vs Exclude (the common mixup)
- Q47. satisfies vs as const vs annotation
- Q48. Typed error handling + Result pattern
- Q49. extends vs intersection &
- Q50. Production Next.js tsconfig

📌 **The one thing to remember above everything else:** TypeScript's job is to catch bugs before runtime. The more specific your types, the earlier TypeScript catches problems. Generics + utility types + discriminated unions = the three pillars of advanced TypeScript.

